Recursion

10 June 2026 06:33

Recursion refers to a programming technique in which a function or method calls itself either directly or indirectly.

Recursive function

```
def A():
  B()
```

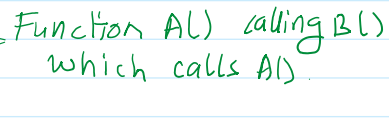
Now if you write a function definition that calls itself, then this function would be known as recursive function.

```
def A():
   A() # Now function A() is called itself from its own function body. So it
  is a recursive function now.
```

Recursion can be indirect also. In the above example, where a function calls itself directly from within its body, is an example of direct recursion. However, if a function calls another function which calls its caller function from within its body, it is known as indirect recursion. For example.

```
def A():
   B()

def B():
   A()
```



So you can say that Recursion can be:

- **Direct recursion:** If a function calls itself directly from its function body.
- **Indirect recursion**: If a function calls another functions which calls its caller function.

  Recursion is used to create a loop by calling the function itself.

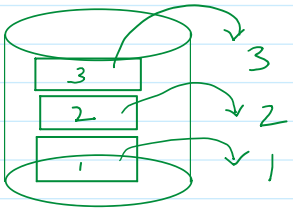
Recursion is a technique for solving a large computational problem by repeatedly applying the same procedures to reduce it to successively smaller problems. A recursive procedure has two parts, 1 or more base cases and a recursive step.

Right or Sensible Recursive code: Is the one that fulfills the following requirements:

- It must have a case. Whose result is known or computed without any recursive calling- **the base case. The base case must be reachable for some argument/parameter.**
- **It also have recursive case where by function calls itself.**

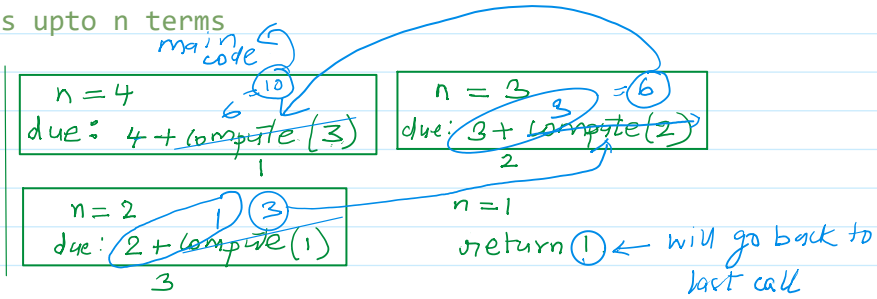
Stack memory follows Last-In-First-Out technique.
The stack memory is used to store the status of method or function during a function call or method call.

The stack memory is used to store the status of method or function during a function call or method call.




```
# Code to find sum of natural numbers upto n terms
# using recursion
def compute(n:int):
    if n == 1:
        return 1
    else:
        return (n + compute(n-1))

n = int(input('Enter number of terms: '))
sum = compute(n)
print(f'Sum of natural numbers upto {n} is {sum}')
```

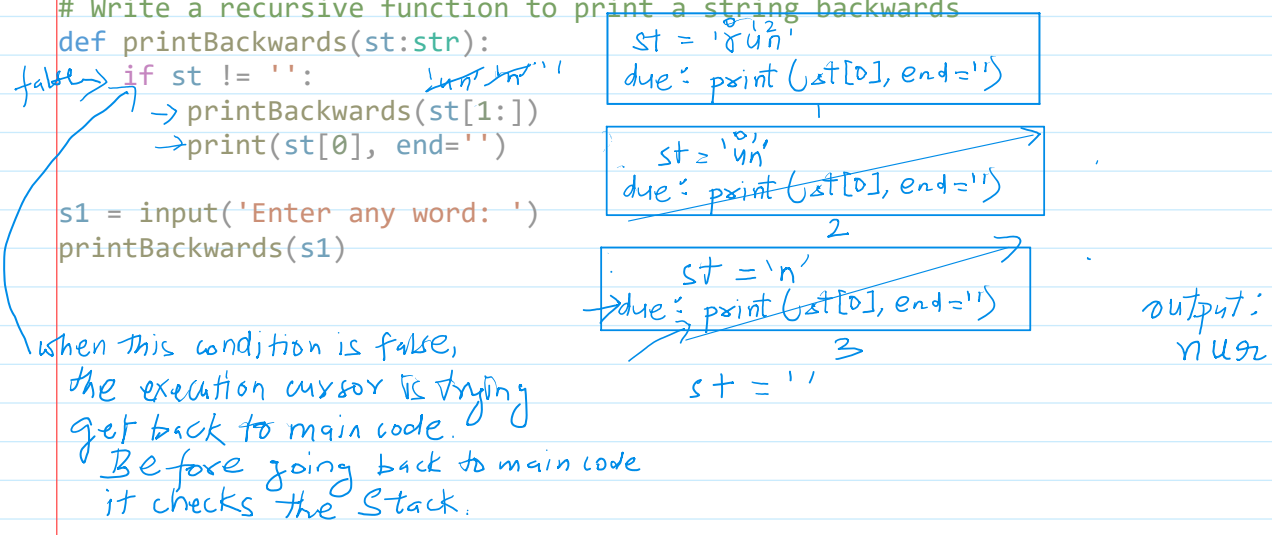


Output:

```
Enter number of terms: 4
Sum of natural numbers upto 4 is 10
```

```
# Write a recursive function to print a string backwards
def printBackwards(st:str):
    if st != '':
        printBackwards(st[1:])
        print(st[0], end='')

s1 = input('Enter any word: ')
printBackwards(s1)
```




```
# power a, b using recursion
def power(a, b):
    if b == 0:
        return 1
    else:
        return a * power(a, b-1)
```

```
#main code
print('Enter only positive numbers.')
a = float(input('Enter the base: '))
b = float(input('Enter raised to the power of: '))
result = power(a, b)
print(f'{a}, raised to the power of {b} is {result}')

Output:
   Enter only positive numbers.
   Enter the base: 4
   Enter raised to the power of: 3
   4.0, raised to the power of 3.0 is 64.0
```

Factorial: 5! = 5 * 4 * 3 * 2 * 1

if n == 0 then its factorial is 1
if n == 1 then its factorial is 1

or

if n==0 or n==1 its factorial is 1

elif n > 1 then its factorial is n*(n-1)*(n-2)…*1

```
# Find factorial of a number using recursion
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n-1)

#main code
n = int(input('Enter any number: '))
f = factorial(n)
print(f'{n}! = {f}')

Output:

   Enter any number: 6
   6! = 720

   Enter any number: 1
   1! = 1

   Enter any number: 0
   0! = 1
```

Home work: Using Recursion

Q1. WAP to input two positive integer values and find GCD of them using recursion.

Q2. WAP to find the sum of all elements present in the list.

Q3. WAP to find sum of digits of number entered by the user.

Q4. WAP to check that given number is Armstrong or not.

153 = 1^3 + 5^3 + 3^3 = 153

Q5. WAP to check that number is lucky number or not. A lucky number is a number in which the eventual sum of the square of the digits of a number is equal to 1.
Example:
28 = 2^2 + 8^2 = 68
68 = 6^2 + 8^2 = 100
100 = 1^2 + 0^2 + 0^2 = 1

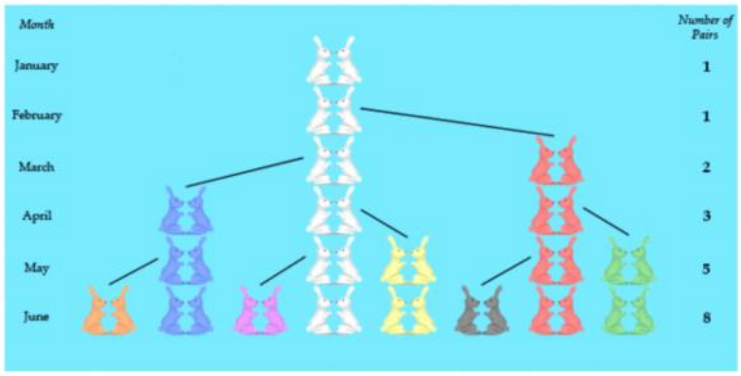
28 is a lucky number

12 = 1^2 + 2^2 = 5
12 is not lucky number

WAP to input number of terms and print all the Fibonacci terms.

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ….
The series is famous for rabbit population.

The puzzle that Fibonacci posed was: how many pairs will there be in one year?



Suppose we let the number of rabbit pairs in the field at the end of the n^{th} month be denoted by F_n.

Consider just the new "baby rabbit pairs" in the n^{th} month. They must be equal in number to the pairs of rabbits that are mature enough to give birth to baby rabbits. This, of course, is precisely the number of rabbit pairs alive two months previously, F_{n-2}.

Now the total number of rabbit pairs in the n^{th} month is the number of pairs alive in the previous month (i.e., F_{n-1}) plus the number of new baby rabbit pairs, F_{n-2}.

Thus, we have the following recursive definition for the n^{th} Fibonacci number,

$$F_0 = F_1 = 1$$ ← base case

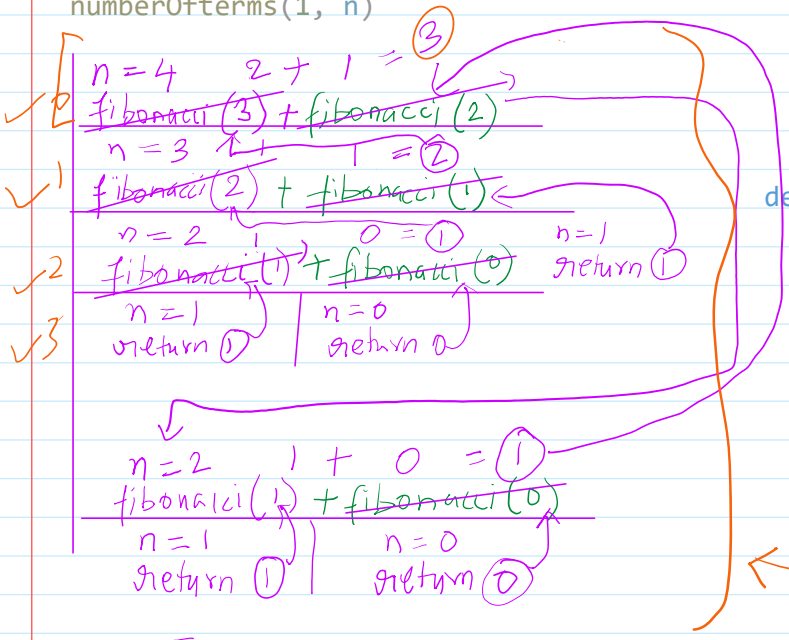
$$F_n := F_{n-1} + F_{n-2}$$ ← recursive case

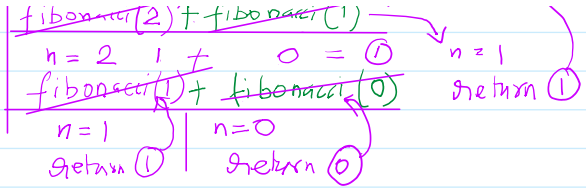
```
def fibonacci(n:int):
    # base cases
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)
```

```
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)

def numberOfterms(start:int, n:int):
    if start <= n:
        print(fibonacci(start))
        numberOfterms(start+1,n)

# main code
n = int(input('Enter number of terms'))
numberOfterms(1, n)
```





Start = 1 2 3 4 5
n = 5
output:
1
1
2
3
5

```
def fibonacci(n:int):
    # base cases
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)
```

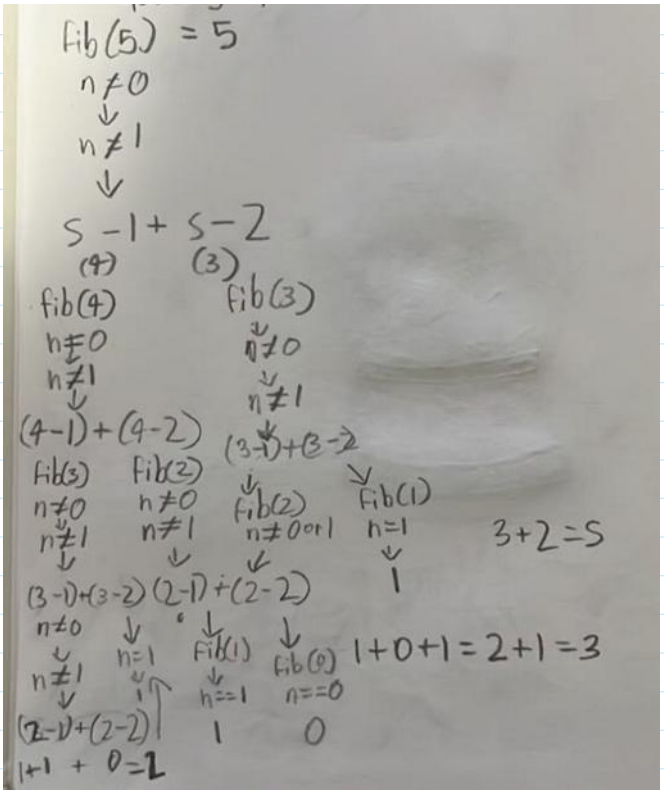
Stack: is a type of list which follows LIFO [Last-in-first-out] is used backtracking algorithm.

Homework

1. Print the first 20 Fibonacci numbers
2. Print Fibonacci numbers between 100 and 1000
3. Find the 50th Fibonacci number
4. Check whether a given number belongs to the Fibonacci series.
5. Print only the even Fibonacci numbers up to a given limit.
6. Calculate the sum of first n Fibonacci numbers.
7. Print the Fibonacci series in reverse order.

0 1 1 2 3 5 8
a b a+b a+b

```
def fibonacci(n:int):
    # base cases
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)
```



Therefore, caller of the method must pass the value for this formal parameter

not containing default value

default values

```
def printfibonacciSeqInReverse(n:int, a = 0, b = 1, i = 1):
    if i < n:
        printfibonacciSeqInReverse(n, b, a+b, i+1)
    print(a if i == 0 else b, end=' ')
# main code
n = int(input('Enter number of terms: '))
printfibonacciSeqInReverse(n)
```